



Connecting to the Cluster and Exposing a Workload

You reach the cluster through a kubeconfig that runs a command to mint a 15-minute AWS token. Users reach your workload because a Service of type LoadBalancer makes the cluster provision an AWS load balancer pointed at your nodes.

IN THIS LECTURE

01 **kubeconfig**

The endpoint, the certificate authority, and the token command.

02 **Creator admin**

Why your first command works and a teammate's is refused.

03 **type: LoadBalancer**

The cloud controller calls AWS and reports back a DNS name.

04 **Built-in vs the controller**

A Classic Load Balancer by default, NLB and ALB with the controller.



A cluster, a user, and a context

The file names the cluster endpoint and its certificate authority, the command that produces credentials, and the context that pairs the two. kubectl reads all three on every call.

~/.kube/config

```
clusters:
- name: demo
  cluster:
    server: https://A1B2C3.gr7.us-east-1.eks.amazonaws.com
    certificate-authority-data: LS0tLS1CRUdJTiBDRVJU

users:
- name: demo-admin
  user:
    exec:
      command: aws
      args: ["eks", "get-token", "--cluster-name", "demo"]

contexts:
- name: demo
  context:
    cluster: demo
    user: demo-admin
    namespace: default

current-context: demo
```



cluster

server + CA

The API server URL, plus the certificate authority the client uses to verify that server's TLS certificate.



user

exec, not a password

On EKS, the command the client runs to fetch a token. No password and no static key is stored here.



context

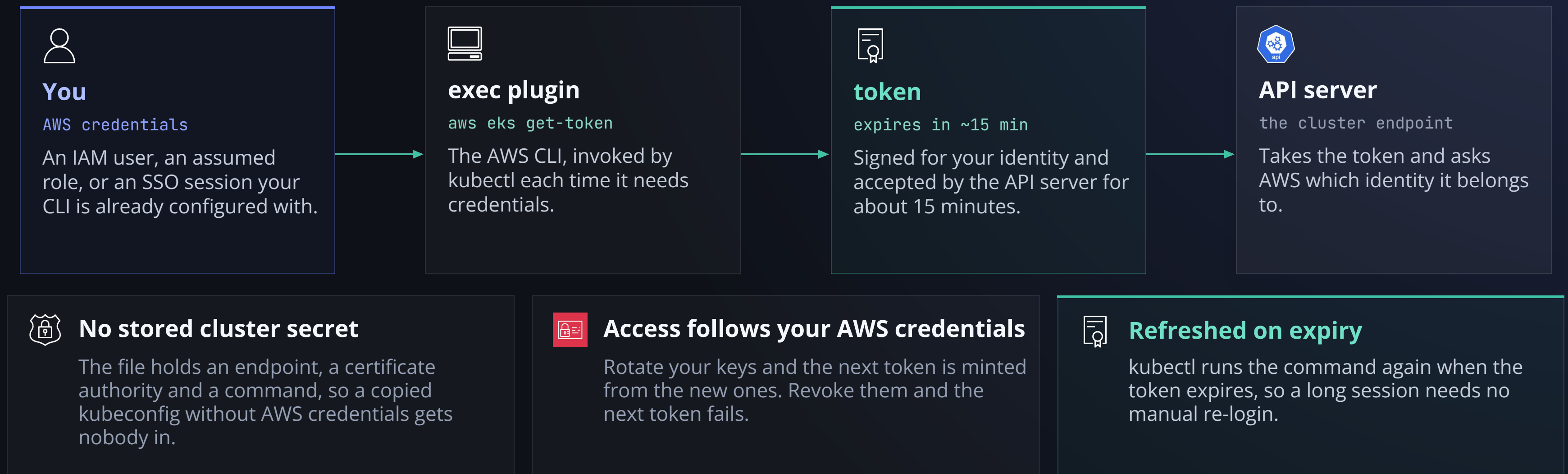
one is current

Pairs a cluster with a user and an optional namespace. The current context is the pairing kubectl uses when you do not name one.

AUTHENTICATION

kubectl mints a short-lived AWS token

The user entry names a command, not a secret. kubectl runs it before a call, gets back a token derived from your AWS credentials that lasts about 15 minutes, and runs it again when that expires.



TWO GATES

Authentication is who you are, authorization is what you may do

The API server first confirms which AWS identity sent the request, then looks up what that identity may do. The second answer comes from a mapping stored in the cluster.



Authentication

who you are

- ✓ Your AWS identity, proven by the short-lived token in the request.
- ✓ Settled between the API server and AWS on every call.
- ! Passing it grants no permission in the cluster.



Authorization

what you may do

- ✓ A mapping from that AWS identity to a set of Kubernetes permissions.
- ✓ Configured in the cluster, not in your kubeconfig.
- ✗ Without an entry for your identity, every action is refused.



Both gates pass

`deployment.apps/web` created

AWS recognizes the token and the mapping permits the verb, so the API server runs the request.



Authenticated, not authorized

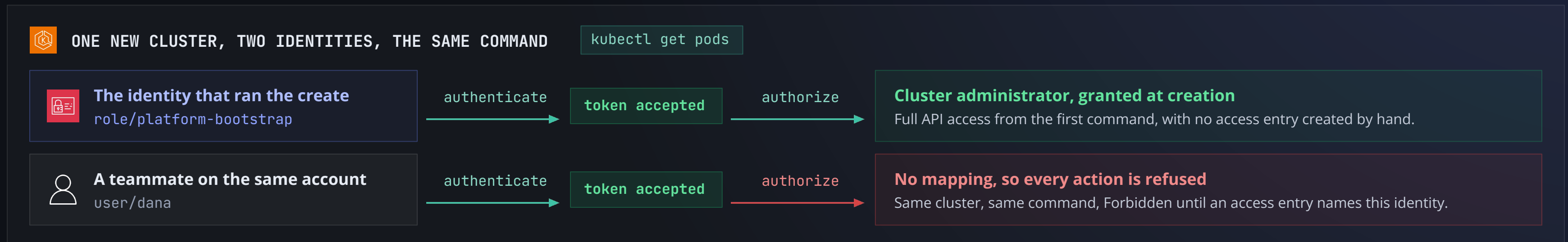
Error from server (Forbidden)

The token was accepted and the mapping was not: you fix this in the cluster access entries, not the kubeconfig.

THE FIRST MAPPING

The cluster creator is granted admin automatically

EKS binds the identity that ran the create to cluster administrator, so the first connection needs no mapping. Every other identity starts with none.



Why it exists

Someone has to be able to connect before any mapping can be created, so EKS settles the first one at creation time.

What it does

Your first kubectl command works with no setup. The same command from a teammate returns Forbidden.

The caveat

It is broad standing access, often bound to an automation role. Add scoped access for real people and treat the grant as a bootstrap.

PROVISIONING

A LoadBalancer Service provisions real AWS infrastructure

The cloud controller calls AWS to create a load balancer pointed at your nodes, then hands its DNS name back as the Service's external address.



zsh

```
$ kubectl get svc web
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)
web	LoadBalancer	10.100.14.9	a1b2c3d4.us-east-1.elb.amazonaws.com	80:30080/TCP



The call comes from inside

No CloudFormation, Terraform or console action created this load balancer. A control plane component called AWS with the cluster IAM role, which is why that role exists.

Built-in path vs the AWS Load Balancer Controller

The cloud controller in the control plane creates a Classic Load Balancer. Install the AWS Load Balancer Controller and a Service creates a Network Load Balancer, an Ingress an Application Load Balancer.



Built-in path

nothing to install

- ✓ A plain LoadBalancer Service is the whole configuration.
- ✓ The cloud controller already in the control plane does the work.
- ✓ Four lines of YAML put a workload on the internet.
- ! Its default type is Classic, which AWS treats as previous generation.



AWS Load Balancer Controller

- ✓ A Service can provision a Network Load Balancer.
- ✓ An Ingress can provision an Application Load Balancer.
- ✓ Host and path routing, TLS termination, far more control.
- ! A deployment you install, grant IAM permissions and keep upgraded.



Classic Load Balancer

What a bare LoadBalancer Service creates. Adequate for a first request, superseded for production.



Network Load Balancer

Layer 4, high throughput, static addresses per zone. The current choice behind a Service.



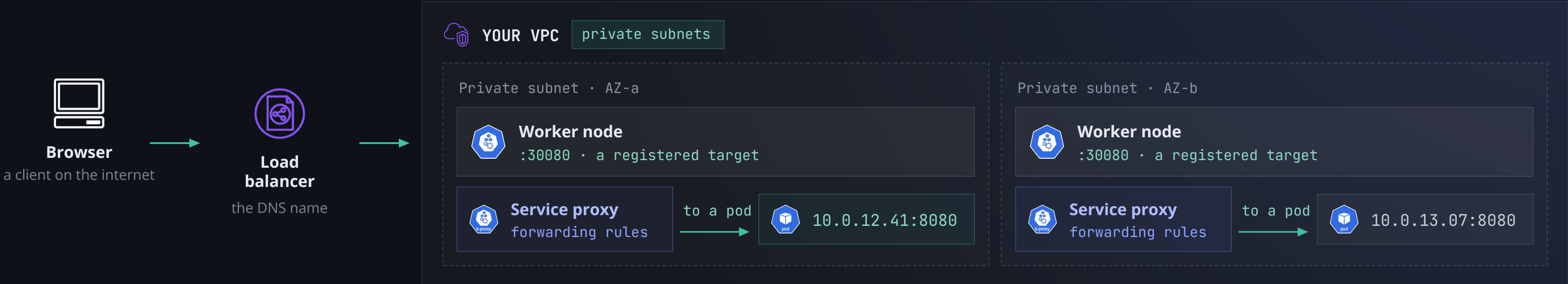
Application Load Balancer

Layer 7, with host and path rules and TLS termination, created from an Ingress.

THE REQUEST PATH

How traffic reaches a pod

The DNS name resolves to the load balancer, which forwards the request to a node on the Service port, where the service proxy picks one of the live pods.



1 Resolve and connect

The browser resolves the load balancer's DNS name and opens a connection to it.

2 Forward to a node

The load balancer picks one of your nodes and sends the request to the Service port.

3 Route to a pod

On that node the service proxy hands the request to one of the live pods.

4 Answer travels back

The pod responds, and the response returns along the same four hops in reverse.